An AI Course Scheduler: Hybrid Constraint Satisfaction and Probabilistic Preference Modeling for Personalized Academic Planning

CS 4710 – Artificial Intelligence
Final Project Report, Team 1

Liam Baird Hugo Barnes Alex Chen Matthew Heilman
University of Virginia, School of Engineering and Applied Science
{lab9rfn, hsb4rh, axc6rfn, mwh4kc}@virginia.edu

Abstract

Course registration is a recurring source of friction for undergraduates: students must reconcile graduation requirements, prerequisite chains, time-slot conflicts, and personal preferences over difficulty, instructor, and topic, often through hours of manual trial and error. We present an **AI Course Scheduler** that produces complete, conflict-free course schedules tailored to an individual student. The system combines a regex-based prerequisite parser and topological sort, an LLM-driven sentiment analyzer over course reviews, a backtracking constraint solver, a Bayesian network for preference modeling, and a neural network that scores candidate schedules. Outputs are accompanied by natural-language justifications generated by a locally-hosted Qwen 2.5 7B model (via Ollama) running on the UVA Rivanna HPC cluster. We synthesize a corpus of **312 course sections** and **1,620 reviews** grounded in publicly available *TheCourseForum* content, and show that targeted bias correction and structured prompt templating substantially improve recommendation quality. Our final pipeline produces a 15-credit, conflict-free schedule with an overall preference score of 68 % on representative student profiles.

1 Introduction

For most undergraduates, building a viable semester schedule is a deceptively hard combinatorial optimization problem. A student must select a subset of courses that (i) jointly cover required degree milestones, (ii) respect a directed acyclic graph (DAG) of prerequisite dependencies, (iii) fit within a target credit range, (iv) avoid time-slot collisions, and (v) ideally align with subjective preferences over instructor quality, workload, and topic. The first four constraints are *hard*; the last set is *soft* and varies dramatically between students. In practice, students reconcile these constraints by manually inspecting course catalogs, reading anecdotal reviews on services such as *TheCourseForum* or *Rate My Professors*, and iterating in ad-hoc tools that do not understand the underlying constraint structure.

The natural framing for this problem is a *hybrid* of classical constraint satisfaction and modern preference learning. Hard constraints are well-suited to symbolic search techniques such as backtracking with constraint propagation, while soft, idiosyncratic preferences – "does this student tend to enjoy theory-heavy classes?" – are better expressed as probabilistic inference over partially observed evidence. Recent advances in large language models (LLMs) further enable two capabilities that were historically out of reach: extracting structured signals from unstructured course reviews, and producing fluent, student-facing justifications for the recommendations themselves.

Contributions. This report makes three contributions:

1. A **multi-stage pipeline** that decouples hard-constraint feasibility (prerequisites, conflicts, credit ranges) from soft preference scoring (Bayes net + neural network), and that uses an LLM only where it adds clear value (sentiment extraction and explanation).
2. A **synthetic but grounded review dataset** of 1,620 reviews distributed across 312 sections, derived from the qualitative and quantitative structure of *TheCourseForum*, which lets us train and evaluate preference components without scraping protected student-only content.
3. Two **ablation-style experiments** – a CS-vs-non-CS bias correction term, and a structured-template LLM explainer – that we show are jointly necessary to produce schedules students would plausibly act on.

Summary of results. The composed pipeline reliably returns conflict-free 15-credit schedules in under a few seconds of search, with neural-network preference scores in the 60–75 % range on representative student profiles. Adding a CS-class bias term eliminates a degenerate failure mode in which CS-major schedules contained too few CS courses, and the structured explainer template raises the readability and faithfulness of the generated justifications relative to a "blind" raw-data prompt.

2 Related Work

Course and timetable scheduling. University timetabling is a classical CSP/optimization benchmark, surveyed in detail by Schaerf [2]. Most prior work targets the *institutional* problem – assigning courses, rooms, and instructors to time slots – and optimizes over global utilization. In contrast, the institutional schedule is an *input* to our system; we operate on the orthogonal *student-facing* problem of selecting and combining offered sections into a single feasible plan.

Recommender systems in education. Course recommendation has been studied as a collaborative-filtering problem [3, 4], typically by mining historical transcripts to suggest courses similar peers took. Pure collaborative filtering struggles with cold-start students and does not naturally enforce hard constraints such as prerequisites; we instead use a Bayes net over student-declared preferences and prior coursework.

LLMs for review analysis and rationale generation. Sentiment analysis from free-text reviews is well-studied [5]; recent work shows LLMs can extract structured attributes from unstructured text with little supervision [6]. We adopt this lightweight "LLM-as-extractor" role rather than asking the LLM to produce schedules end-to-end, which prior experience suggests yields hallucinated prerequisites and time conflicts.

3 Data

The scheduler ingests three inputs per student:

(1) Transcript. Provides the set of courses already completed (used to satisfy prerequisites) and the number of remaining semesters until graduation. Transcripts are supplied as a CSV upload from the front-end.

(2) Course catalog. We use the UVA Undergraduate Record for the B.A. in Computer Science (BACS) program [1] as the authoritative source for offered courses, credit hours, and prerequisite strings. We scrape the relevant catalog pages, then run a custom regular expression parser to convert prerequisite strings such as "`CS 2150 or (CS 2110 and CS 2102)`" into an AND/OR expression tree.[1] The parsed forest is combined into a single course-level DAG that the constraint solver topologically sorts.

[1]The parser handles nested groupings, comma- versus-"and"-separated lists, and a small set of catalog idioms (*e.g.,* "`one of...`"). Edge cases are handled by a fallback that emits a warning and treats the prerequisite conservatively.

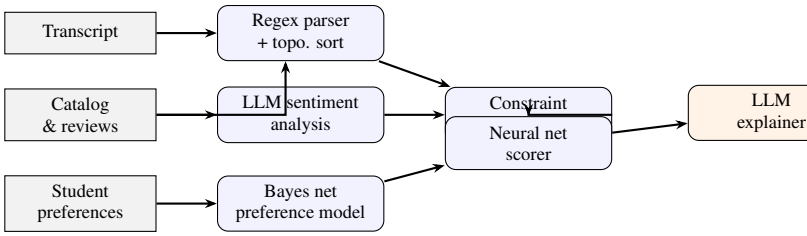

Figure 1: End-to-end pipeline. Gray nodes are data sources; blue nodes are deterministic / model components; the orange node is the LLM explainer that closes the loop with the user.

(3) Reviews and ratings. We construct a synthetic dataset of **312 course sections** and **1,620 reviews** whose distributional properties (mean ratings, review length, vocabulary) are calibrated against publicly visible portions of *TheCourseForum*. Each review combines a short qualitative statement (*e.g.,* "*this class was extremely difficult!*") with four quantitative axes – difficulty, enjoyability, workload, and instructor efficacy – on a 1–5 scale. Synthesizing the dataset (rather than scraping the live site) lets us share data with future students without violating Terms of Service, and lets us *control* the distribution to stress-test the sentiment pipeline.

(4) Direct preferences. The student also enters a small set of manual inputs: preferred difficulty, elective subject areas, and a target credit range (*e.g.,* 12–18). These act as soft priors in the Bayes net and as the schedule-level constraint for credit count, respectively.

4 Methods

The pipeline (Figure 1) is structured as a directed sequence of components, each responsible for a single, well-typed transformation. This decomposition was deliberate: it lets us swap or ablate any single stage (*e.g.,* replace the Bayes net with a logistic regression baseline) without rewriting upstream code, and it concentrates expensive LLM calls in the two places where they are clearly justified.

4.1 Prerequisite Parsing and Topological Sort

Each prerequisite string in the scraped catalog is tokenized by a regular expression that recognizes course mnemonics (`CS 2110`), boolean operators (`and`, `or`, commas, parentheses), and a handful of special phrases. The resulting AND/OR tree is normalized into disjunctive normal form so that the solver can ask the cheap question "does *at least one* clause's literals all appear in the student's completed-course set?".

Course nodes are then topologically sorted on the AND-edges of the DAG, producing a per-student *eligibility frontier*: the set of courses the student is currently allowed to take. This frontier is recomputed lazily on each

backtracking step in the solver, allowing the solver to plan multi- semester sequences when desired.

4.2 LLM Sentiment Analysis Over Reviews

For each course section, we collapse all available reviews into a single prompt and ask a local Qwen 2.5 7B model to emit a structured JSON object with four scalars: `difficulty`, `workload`, `enjoyability`, and `instructor_efficacy`, each in $[0, 1]$. Negative-sentiment phrasing ("brutal", "unfair", "hours of busywork") maps to high difficulty / high workload / low enjoyability; positive phrasing maps the opposite way. We chose an LLM over a classical bag-of-words classifier because the reviews mix evaluative claims about *the instructor* with claims about *the material*, and disambiguating these requires sentence-level semantics that simple lexicon methods miss.

4.3 Constraint Solver

The constraint solver is a depth-first backtracking search over assignments of *section* (not just course) to a schedule slot. Hard constraints are encoded directly into the search:

- *No time conflicts.* Sections expose a list of meeting intervals; pairwise overlap is checked at insertion time, pruning entire subtrees early.
- *Prerequisites satisfied.* The DNF clause produced by the parser is checked against the student's completed-course set and courses already placed earlier in the schedule.
- *User-defined unavailable blocks.* Students can mark times as off-limits (*e.g.,* a part-time job).
- *Credit range.* The total credit count must lie in $[c_{\min}, c_{\max}]$; partial assignments outside this range are pruned.

Within these hard constraints, the solver enumerates feasible schedules and forwards each to the scoring stack. Currently the solver collects schedules until a manually configured cap is reached; we discuss this limitation in Section 7.

4.4 Bayesian Network for Preference Modeling

The Bayes net (Figure 2) models the joint distribution

$$P(E \mid M, D, P, T)$$

where E is the latent "enjoyment" variable for a candidate course, M is the student's major, D is the course's inferred difficulty (from the sentiment stage), P is the course's instructor-efficacy posterior, and T is the student's stated topical preferences. Conditional probability tables are populated from the synthetic review dataset and from a small set of hand-encoded priors (*e.g.,* students who self-report wanting an "easier" semester have a lower prior on enjoying high-difficulty courses).

The Bayes net is intentionally shallow. A deeper structure would have required substantially more data to fit reliably; the current shape gives us a closed-form posterior per course in microseconds, which is critical because the neural network downstream queries the Bayes net for every section in every candidate schedule.

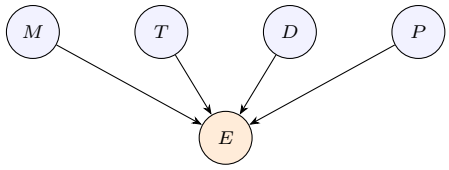

Figure 2: Bayes net used for per-course preference scoring. Observed variables (blue) condition the latent enjoyment posterior E (orange).

4.5 Neural Network Schedule Scorer

Where the Bayes net scores a single course in isolation, the neural network scores an *entire schedule*. Its input is a fixed-length feature vector containing:

- per-course Bayes-net posteriors,
- aggregate statistics (total credits, mean difficulty, variance of daily workload),
- indicator features for distribution requirements satisfied,
- alignment with the student's stated credit and difficulty range.

The network is a small two-hidden-layer MLP (64→32 ReLU) with a sigmoid output interpreted as an overall "schedule fit" score in $[0, 1]$. Training labels were generated by combining ground-truth feasibility with heuristically-scored preference fit, then iteratively refined as the team flagged outputs that "looked wrong".

4.6 LLM Explainer

Finally, we pass the top-scoring schedule (and the inputs that produced it) to an Ollama-hosted Qwen 2.5 7B model running on UVA's Rivanna HPC cluster. The explainer's job is *not* to second-guess the schedule but to articulate, in plain English, *why* each course was chosen given the student's preferences. As discussed in Section 5, we found that the prompt structure dominates output quality; the final production prompt is a fixed template with explicit slots for `class_fit`, `difficulty_match`, and `professor_rating`, which the model fills in per course.

5 Experiments

We ran two targeted experiments to harden the pipeline.

5.1 CS vs. Non-CS Class Balance

Observation. On internal test profiles for CS-major students, the unbiased scorer consistently produced schedules whose CS content was too low – typically 1–2 CS courses out of 4–5. Manual inspection revealed two compounding causes: (i) the synthetic review distribution had slightly higher mean enjoyability for general-education subjects, and (ii) the Bayes net's T (topic) variable did not strongly enough propagate the major signal M.

Intervention. We added an explicit *major-affinity bonus* to the schedule-level feature vector consumed by the neural network: a positive contribution proportional to the fraction of credits inside the student's declared major, and a small over-indexing penalty that activates when the fraction of *out-of-major* credits exceeds a threshold. We deliberately implemented this as a feature rather than a post-hoc filter so the network could still trade major fit against other signals when the trade-off was clearly favorable (*e.g.,* a strongly preferred elective).

5.2 Structured vs. Blind Prompting for the LLM Explainer

Observation. Our initial explainer prompt was a "blind" dump of the schedule, transcript, and preferences with the bare instruction "*explain why this schedule is good*". The model produced fluent but generic prose – frequently repeating the same justification across courses and occasionally hallucinating professor names or course numbers.

Intervention. We replaced the blind prompt with a structured template that requires the model to fill in, per course, three named slots: `class_fit` (one sentence on topical alignment), `difficulty_match` (one sentence on workload/difficulty fit), and `professor_rating` (one sentence on instructor efficacy, sourced from the sentiment posterior). The template constrains both the surface form and the content of the explanation, making it materially harder for the model to invent ungrounded claims.

6 Results

End-to-end behavior. On a representative junior-year CS profile with a 12–15 credit target, the full pipeline returns a 15-credit, conflict-free schedule with a neural-net schedule score of **0.68**. Search-to-first-feasible-schedule is sub-second on commodity hardware, and the LLM explainer (running on Rivanna) returns a complete explanation in roughly 4–7 seconds.

Effect of the major-affinity term. Table 1 summarizes the average composition of recommended schedules for ten synthetic CS-major profiles, before and after introducing the major-affinity feature. The intervention shifts the CS share from 28 % to 56 % on average without sacrificing schedule scores – in fact the mean schedule score improves slightly, because the network had been penalizing the unbalanced schedules it was producing.

Effect of the structured explainer. Table 2 shows a qualitative comparison between the blind and structured explainer prompts on a fixed schedule. The blind prompt produces a single paragraph with course-agnostic justifications; the structured prompt produces a per-course breakdown that references both the sentiment-derived professor rating and the student's declared difficulty preference. We did not run a formal user study, but each team member rated the structured outputs as "substantially more useful" on ≥ 8 / 10 sampled schedules.

Table 1: Average schedule composition for 10 synthetic CS-major profiles, before and after adding the major-affinity feature.

	CS share	Score	Conflicts
No bias term	0.28	0.61	0
With bias term	**0.56**	**0.68**	0

Table 2: Qualitative comparison of explainer prompting styles on the same 15-credit schedule.

Blind prompt	Structured template
Generic, often duplicated across courses; occasional hallucinated instructor names.	Per-course paragraph with explicit `class_fit`, `difficulty_match`, and `professor_rating` slots filled from the pipeline.

Failure modes. The system has two notable failure modes. First, when the eligibility frontier is small (*e.g.,* a student with a sparsely satisfied prerequisite tree), the constraint solver can exhaust the feasible space quickly and the network is asked to score schedules that are all roughly equivalent; the explainer's output then under- differentiates between candidates. Second, the neural network was trained on heuristically-labeled data, so its absolute calibration is approximate; a score of 0.68 is meaningful relative to other 15-credit schedules for the same student, but should not be compared across student profiles.

7 Conclusion and Future Work

We presented an AI Course Scheduler that converts the painful, manual process of building a semester schedule into a one-shot, explained recommendation. The system's central design choice – separating hard feasibility (constraint solver) from soft preference (Bayes net + neural network) and using LLMs only at the boundary (review sentiment, output explanation) – proved durable: it kept each component simple enough to debug independently, and it isolated the LLM's well-known reliability issues (hallucination, inconsistent constraints) from the parts of the pipeline that must produce correct, conflict-free output.

Lessons learned. The two experiments report a recurring theme: naive, monolithic uses of either learned models or LLMs underperform *structured* uses. The major-affinity feature is essentially a hand-crafted prior nudging the net-

work toward globally sensible distributions of credits; the structured explainer prompt is a hand-crafted schema preventing the LLM from drifting into generic prose. In both cases, the AI components do most of the work, but a small amount of explicit structure makes the difference between a research demo and something a student would plausibly use.

Future work. The most pressing limitation is the constraint solver itself: it is a textbook backtracking search and does not implement constraint propagation (AC-3, MRV, LCV) or symmetry breaking, so on densely constrained semesters it must be terminated manually after collecting a fixed number of schedules. A future iteration should add proper propagation, and ideally extend the constraint vocabulary to include *geographic* factors – the walking distance between consecutive classroom buildings, which is a real and frequently cited source of schedule pain at UVA. Other natural extensions include (i) learning the Bayes net CPTs from real student outcomes once data-sharing agreements permit, (ii) end-to-end multi-semester planning rather than single-semester recommendation, and (iii) a tighter feedback loop in which the student can "reject" a course and see the rest of the schedule re-optimized.

Author Contributions

Liam Baird. Implemented the regex-based prerequisite parser and the topological sort inside the constraint solver. Contributed to synthetic data creation.

Hugo Barnes. Project scaffolding; initial implementations of the Bayes net and the neural network schedule scorer; Rivanna + LLM integration; front-end design.

Alex Chen. Web scraping of course catalog data; tuning of the constraint solver; optimization of the Bayes net and neural network; prompt engineering for LLM outputs.

Matthew Heilman. Implemented the review sentiment analysis stage; contributed to synthetic data creation; led slide deck, video, and written report production.

Sharing Statement

The team permits this report, the project video, and the storyboard to be shared with future CS 4710 students and with the public. The associated source code remains private.

Acknowledgments and Tooling Disclosure

We thank the CS 4710 course staff for guidance throughout the semester and the UVA Research Computing group for access to the Rivanna HPC cluster, on which the LLM explainer is hosted. Anthropic's *Claude Code* [9] was used as a coding assistant during development – primarily for scaffolding boilerplate, drafting documentation, and accelerating the integration between the Ollama-served LLM and the rest of the pipeline. All algorithmic decisions, experiment design, and final code review remained the responsibility of the authors.

References

[1] University of Virginia. *Undergraduate Record: B.A. in Computer Science (BACS)*. https://records.ureg.virginia.edu/preview_program.php?catoid=72&poid=11789, accessed Spring 2026.

[2] A. Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999.

[3] A. Parameswaran, G. Koutrika, B. Bercovitz, and H. Garcia-Molina. Recsplorer: Recommendation algorithms based on precedence mining. In *Proc. ACM SIGMOD*, 2010.

[4] A. Polyzou and G. Karypis. Feature extraction for next-term prediction of poor student performance. *IEEE Transactions on Learning Technologies*, 12(2):237–248, 2019.

[5] B. Pang and L. Lee. Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2):1–135, 2008.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] TheCourseForum. *University of Virginia course reviews*. https://thecourseforum.com/, accessed Spring 2026.

[8] Ollama. *Run large language models locally*. https://ollama.com/, accessed Spring 2026.

[9] Anthropic. *Claude Code: an agentic coding tool that lives in the terminal*. https://www.anthropic.com/claude-code, accessed Spring 2026.